APÉNDICE I. PROGRAMA MAT-FEM CON RMA

MAT-fem es un programa libre de código abierto, por lo que la implementación del procedimiento de refinamiento de malla adaptativo también lo es. A continuación se adjuntan el programa modificado con todas las funciones para su consulta.

A. Programa MATfem.m

```
%% MAT-fem

% Clear memory and variables
  clear

% The variables are read as a MAT-fem subroutine
% pstrs = 1 indicate Plane Stress; 0 indicate Plane Strain
% young =   Young Modulus
% poiss =   Poission Ratio
% thick =   thickness only valid for Plane Stress
% denss =   density
% coordinates = [ x , y ] coordinate matrix nnode x ndime (2)
% elements    = [ inode, jnode, knode ] element connectivity  matrix
%               nelem x nnode; nnode = 3 for triangular elements and
%               nnode = 4 for quadrilateral elements
% fixnodes    = [node number, dimension, fixed value] matrix with
%               Dirichlet restrictions.
% pointload   = [node number, dimension, load value] matrix with
%               nodal loads.
% SideLoad    = [node number i, node number j, x load, y load] matrix with
%               line definition and uniform load applied in x and y
%               directions

  file_name = input('Enter the file name: ','s');

%%% Elección del criterio de malla óptima
% Criterios disponibles:
% 1. Equi-distribución de error global
% 2. Equi-distribución de error específico
  cmo = 1;
% Porcentaje de error máximo permisible
  maxerror = 0.05;      % Error permisible del 5%
%%%

  tic;                   % Start clock
  ttim = 0;              % Initialize time counter
  eval (file_name);      % Read input file

% Finds basics dimensions
  npnod  = size(coordinates,1);        % Number of nodes
  nndof  = 2*npnod;                    % Number of total DOF
  nelem  = size(elements,1);           % Number of elements
  nnode  = size(elements,2);           % Number of nodes per element
  neleq  = nnode*2;                    % Number of DOF per element

  ttim = timing('Time needed to read the input file',ttim); %Reporting time
```

```
% Dimension the global matrices
  StifMat = sparse ( nndof , nndof );  % Create the global stiffness matrix
  force   = sparse ( nndof , 1 );      % Create the global force vector

% Material properties (Constant over the domain)
  dmat = constt(young,poiss,pstrs);

  ttim = timing('Time needed to  set initial values',ttim); %Reporting time

% Element loop
  ElemDim = []; %%%

  for ielem = 1 : nelem

% Recover element properties
    lnods = elements(ielem,:);                        % Elem. connectivity
    coord(1:nnode,:) = coordinates(lnods(1:nnode),:); % Elem. coordinates

%%% Obtención del tamaño de los elementos
    if nnode == 3
        ElemDim(ielem,1) = min([norm(coord(1,:)-coord(2,:)) ...
                                norm(coord(2,:)-coord(3,:)) ...
                                norm(coord(3,:)-coord(1,:))]);
    else
        ElemDim(ielem,1) = min([norm(coord(1,:)-coord(2,:)) ...
                                norm(coord(2,:)-coord(3,:)) ...
                                norm(coord(3,:)-coord(4,:)) ...
                                norm(coord(4,:)-coord(1,:))]);
    end
%%%

% Evaluates the elemental stiffness matrix and mass force vector
    if (nnode == 3)
      [ElemMat,ElemFor] = TrStif(coord,dmat,thick,denss); % 3 Nds Triangle
    else
      [ElemMat,ElemFor] = QdStif(coord,dmat,thick,denss); % 4 Nds Quad.
    end

% Finds the equation number list for the i-th element
    eqnum = [];                                  % Clear the list
    for i =1 : nnode                             % Node cicle
      eqnum = [eqnum,lnods(i)*2-1,lnods(i)*2];   % Build the equation
    end                                          % number list

% Assemble the force vector and the stiffness matrix
    for i = 1 : neleq
      force(eqnum(i)) = force(eqnum(i)) + ElemFor(i);
      for j = 1 : neleq
         StifMat(eqnum(i),eqnum(j)) = StifMat(eqnum(i),eqnum(j)) + ...
                                      ElemMat(i,j);
      end
    end

  end  % End element loop

  ttim = timing('Time to assamble the global system',ttim); %Reporting time

% Add side forces to the force vector
  for i = 1 : size(sideload,1)
      x=coordinates(sideload(i,1),:)-coordinates(sideload(i,2),:);
      l = sqrt(x*transpose(x));      % Finds the length of the side
      ieqn = sideload(i,1)*2;        % Finds eq. number for the first node
      force(ieqn-1) = force(ieqn-1) + l*sideload(i,3)/2;    % add x force
      force(ieqn  ) = force(ieqn  ) + l*sideload(i,4)/2;    % add y force

      ieqn = sideload(i,2)*2;        % Finds eq. number for the second node
      force(ieqn-1) = force(ieqn-1) + l*sideload(i,3)/2;    % add x force
      force(ieqn  ) = force(ieqn  ) + l*sideload(i,4)/2;    % add y force
```

```
  end

% Add point loads conditions to the force vector
  for i = 1 : size(pointload,1)
      ieqn = (pointload(i,1)-1)*2 + pointload(i,2);     % Finds eq. number
      force(ieqn) = force(ieqn) + pointload(i,3);       % add the force
  end

  ttim = timing('Time for apply side and point load',ttim); %Reporting time

% Applies the Dirichlet conditions and adjust the right hand side
  u = sparse ( nndof, 1 );
  for i = 1 : size(fixnodes,1)
    ieqn = (fixnodes(i,1)-1)*2 + fixnodes(i,2); % Finds the equation number
    u(ieqn) = fixnodes(i,3);                    % and store the solution in u
    fix(i) = ieqn;                          % and mark the eq as a fix value
  end
  force = force - StifMat * u;      % Adjust the rhs with the known values

% Compute the solution by solving StifMat * u = force for the
% remaining unknown values of u.
  FreeNodes = setdiff ( 1:nndof, fix );     % Finds the free node list and
                                            % solve for it.
  u(FreeNodes) = StifMat(FreeNodes,FreeNodes) \ force(FreeNodes);

% Compute the reactions on the fixed nodes as a R = StifMat * u - F
  reaction = sparse(nndof,1);
  reaction(fix) = StifMat(fix,1:nndof) * u(1:nndof) - force(fix);

  ttim = timing('Time to  solve the stifness matrix',ttim); %Reporting time

% Compute the stresses
  [Strnod,Struns,EVol] = Stress(dmat,poiss,thick,pstrs,u); %%%

  ttim = timing('Time to  solve the  nodal stresses',ttim); %Reporting time

%%% Estimación del error y cálculo de la energía de deformación
  [EError,EU] = ErrEst(dmat,thick,pstrs,Strnod,Struns);
  GlobalError = sqrt(transpose(EError)*EError);
  GlobalU = sqrt(transpose(EU)*EU);

  ttim = timing('Time to estimate the element error',ttim); %Reporting time
%%%

%%% Refinamiento de malla
  [ElemRef,NewElemSize] = Refinement(cmo,EError,GlobalError,maxerror,...
                                     GlobalU,EVol,ElemDim);
  GlobalRef = GlobalError/(maxerror*GlobalU); % Parámetro global de ref.

  ttim = timing('Time to  define  new element sizes',ttim); %Reporting time
%%%

%%% Representación gráfica de los resultados y generación de la nueva malla
  ToGiD (file_name,u,reaction,Strnod,ElemRef,NewElemSize);

  ttim = timing('Time used to  write  the  solution',ttim); %Reporting time
%%%

%%% Informe de algunos parámetros de interés
  fprintf('\n\t      Global Error = %0.3f',GlobalError);
  fprintf('\n\t     Allowed Error = %0.2f',maxerror);
  fprintf('\n\t          Global U = %0.3f',GlobalU);
  fprintf('\n\t   Global Ref Coef = %0.3f',GlobalRef);
  fprintf('\n\tNumber of Elements = %0.0f \n',nelem);
%%%

  itim = toc;                                               %Close last tic
  fprintf(1,'\nTotal running time %12.6f \n\n',ttim); %Reporting final time
```

B. Función constt.m

```
function D = constt (young,poiss,pstrs)

%% constt Evaluates the constitutive matrix.
%
%  Parameters:
%
%    Input, Young: Young modulus.
%           poiss: Poisson ratio
%           pstrs: flag for Plane Stress
%
%    Output, D the element constitutive matrix
%

%  Plane Sress
   if (pstrs==1)
       aux1 = young/(1-poiss^2);
       aux2 = poiss*aux1;
       aux3 = young/2/(1+poiss);
%  Plane Strain
   else
       aux1 = young*(1-poiss)/(1+poiss)/(1-2*poiss);
       aux2 = aux1*poiss/(1-poiss);
       aux3 = young/2/(1+poiss);
       thick= 1.0;
   end

   D = [aux1,aux2,0;aux2,aux1,0;0,0,aux3];
```

C. Función timing.m

```
function t = timing (text,time)

%% timing Evaluates and accumulate the used time.
%
%  Parameters:
%
%    Input, text : String to show.
%           time : Accumulate time at t-1
%
%    Output, t   : Accumulate time at t

  itim = toc;                             % Close previous tic
  fprintf(1,[text,' %12.6f \n'],itim);  % Output time and text
  t = itim + time;                        % Accumulate time in t

  tic                                     % Open a new tic
```

D. Función TrStif.m

```
function [M,F] = TrStif (nodes,dmat,thick,denss)

%% TrStif Evaluates the stiffness matrix for a triangular element.
%
%  Parameters:
%
%    Input, nodes: contains the 2D coordinates nodes coordinates
%                  of the vertices and the Element thickness.
%           dmat : constitutive matrix
%           denss and thick : density and thickness
%
%    Output, M the element local stiffness matrix
%            F the element local force vector


  b(1) = nodes(2,2) - nodes(3,2);
  b(2) = nodes(3,2) - nodes(1,2);
  b(3) = nodes(1,2) - nodes(2,2);

  c(1) = nodes(3,1) - nodes(2,1);
  c(2) = nodes(1,1) - nodes(3,1);
  c(3) = nodes(2,1) - nodes(1,1);

  area2 = abs(b(1)*c(2) - b(2)*c(1));
  area = area2 / 2;

  bmat = [b(1),  0 ,b(2),  0 ,b(3),  0 ;
            0 ,c(1),  0 ,c(2),  0 ,c(3);
           c(1),b(1),c(2),b(2),c(3),b(3)];

  bmat = bmat / area2;

  M = (transpose(bmat)*dmat*bmat)*area*thick;

  force = area*denss*thick/3;

  F = [0,-force,0,-force,0,-force];
```

E. Función QdStif.m

```
function [M,F] = QdStif (nodes,dmat,thick,denss)

%% QdStif Evaluates the stiffness matrix for a quadrilateral element.
%
%  Parameters:
%
%    Input, nodes: contains the 2D coordinates nodes coordinates
%                  of the vertices and the Element thickness.
%           dmat : constitutive matrix
%           denss and thick : density and thickness
%
%    Output, M the element local stiffness matrix
%            F the element local force vector

  fform = @(s,t)[(1-s-t+s*t)/4,(1+s-t-s*t)/4,(1+s+t+s*t)/4,(1-s+t-s*t)/4];
  deriv = @(s,t)[(-1+t)/4,( 1-t)/4,( 1+t)/4,(-1-t)/4 ;
                 (-1+s)/4,(-1-s)/4,( 1+s)/4,( 1-s)/4 ];

  pospg = [ -0.577350269189626E+00 , 0.577350269189626E+00 ];
  pespg = [  1.0E+00 , 1.0E+00];
  M = zeros(8,8);
  fy = zeros(1,4);

  for i=1 : 2
     for j=1 : 2
        lcffm = fform(pospg(i),pospg(j)) ;    % FF at gauss point
        lcder = deriv(pospg(i),pospg(j)) ;    % FF Local derivatives
        xjacm = lcder*nodes ;                 % Jacobian matrix
        ctder = xjacm\lcder ;                 % FF Cartesian derivates
        darea = det(xjacm)*pespg(i)*pespg(j)*thick;

        bmat = [];
        for inode = 1 : 4
          bmat = [ bmat , [ctder(1,inode),            0 ;
                                        0 ,ctder(2,inode);
                           ctder(2,inode),ctder(1,inode) ] ] ;
        end

        M = M + (transpose(bmat)*dmat*bmat)*darea;

        fy = fy + lcffm*denss*darea;

     end
  end

  F = [ 0, -fy(1), 0, -fy(2), 0, -fy(3), 0, -fy(4)];
```

F. Función Stress.m

```
function [S,C,V] = Stress (dmat,poiss,thick,pstrs,u)

%% Stress Evaluates the stresses at the gauss point and smooth the values
%         to the nodes.
%
%  Parameters:
%
%    Input, nodes: Contains the 2D coordinates nodes coordinates
%                   of the vertices and the Element thickness.
%             dmat: Constitutive matrix
%            poiss: Poisson ratio
%            thick: Thickness
%            pstrs: Flag for Plane Stress
%                u: Nodal displacements
%
%    Output, S the nodal stress matrix (nnode, nstrs)
%            C the nodal unsmoothed stress matrix
%            V the element volumes vector
%
  global coordinates;
  global elements;

  nelem  = size(elements,1);           % Number of elements
  nnode  = size(elements,2);           % Number of nodes per element
  npnod  = size(coordinates,1);        % Number of nodes

  if (pstrs==1)
      nstrs= 3;                        % Number of Strs. Sx Sy Txy
  else
      nstrs= 4;                        % Number of Strs. Sx Sy Sz Txy
  end

  nodstr = zeros(npnod,nstrs+1);
  C = []; %%%
  V = []; %%%
  for ielem = 1 : nelem

% Recover element properties
    lnods = elements(ielem,:);
    coord(1:nnode,:) = coordinates(lnods(1:nnode),:);
    eqnum = [];
    for i =1 : nnode
      eqnum = [eqnum,lnods(i)*2-1,lnods(i)*2];
    end
    displ = u(eqnum);

% Stresses inside the elements.
    if (nnode == 3)
% Triangular elements constant stress
      [ElemStr,earea] = TrStrs(coord,dmat,displ,poiss,thick,pstrs); %%%
      for j=1 : nstrs
        nodstr(lnods,j) = nodstr(lnods,j) + ElemStr(j);
      end
      nodstr(lnods,nstrs+1) = nodstr(lnods,nstrs+1) + 1;
    else
% Quadrilateral elements stress at nodes
      [ElemStr,earea] = QdStrs(coord,dmat,displ,poiss,thick,pstrs); %%%
      for j=1 : 4
        for i = 1 : nstrs
          nodstr(lnods(j),i) = nodstr(lnods(j),i) + ElemStr(i,j);
        end
      end
      nodstr(lnods,nstrs+1) = nodstr(lnods,nstrs+1) + 1;
    end
```

```
%%% Almacenamos las tensiones sin alisar y los volúmenes de los elementos
    C = [C;transpose(ElemStr)];
    V = [V;earea*thick];
%%%
  end
% Find the mean stress value
  S = [];
  for i = 1 : npnod
      S = [S ; nodstr(i,1:nstrs)/nodstr(i,nstrs+1)];
  end
```

G. Función TrStrs.m

```
function [S,area] = TrStrs (nodes,dmat,displ,poiss,thick,pstrs)

%% TriStrs Evaluates the stresses for a triangular element.
%
%  Parameters:
%
%    Input, nodes: contains the 2D coordinates nodes coordinates
%                  of the vertices and the Element thickness.
%             dmat: constitutive matrix
%           displ: nodal displacement
%           poiss: Poisson ratio
%           thick: thickness
%           pstrs: flag for Plane Stress
%
%    Output, S the element constant stress vector
%         area the element area
%

  b(1) = nodes(2,2) - nodes(3,2);
  b(2) = nodes(3,2) - nodes(1,2);
  b(3) = nodes(1,2) - nodes(2,2);

  c(1) = nodes(3,1) - nodes(2,1);
  c(2) = nodes(1,1) - nodes(3,1);
  c(3) = nodes(2,1) - nodes(1,1);

  area2 = abs(b(1)*c(2) - b(2)*c(1));
  area = area2 / 2;

  bmat = [b(1),  0 ,b(2),  0 ,b(3),  0 ;
            0 ,c(1),  0 ,c(2),  0 ,c(3);
          c(1),b(1),c(2),b(2),c(3),b(3)];

  se = (dmat*bmat*displ)/area2;

%  Plane Stress
  if (pstrs==1)
       S = se ;
%  Plane Strain
   else
       S = [se(1);se(2);-poiss*(se(1)+se(2));se(3)];
   end
```

H. Función QdStrs.m

```
function [S,area] = QdStrs (nodes,dmat,displ,poiss,thick,pstrs)

%% QdStrs Evaluates the stresses for a quadrilateral element.
%
%  Parameters:
%
%    Input, nodes: contains the 2D coordinates nodes coordinates
%                  of the vertices and the Element thickness.
%             dmat: constitutive matrix
%           displ: nodal displacement
%           poiss: Poisson ratio
%           thick: thickness
%           pstrs: flag for Plane Stress
%
%    Output, S the element constant stress vector
%         area the element area
%

%%% Cálculo del área del elemento
    c(1) = nodes(2,2) - nodes(3,2);
    c(2) = nodes(3,2) - nodes(1,2);

    d(1) = nodes(3,1) - nodes(2,1);
    d(2) = nodes(1,1) - nodes(3,1);

    e(1) = nodes(3,2) - nodes(4,2);
    e(2) = nodes(4,2) - nodes(1,2);

    f(1) = nodes(4,1) - nodes(3,1);
    f(2) = nodes(1,1) - nodes(4,1);

    area2 = abs(c(1)*d(2) - c(2)*d(1)) + abs(e(1)*f(2) - e(2)*f(1));
    area = area2 / 2;
%%%

   fform = @(s,t)[(1-s-t+s*t)/4,(1+s-t-s*t)/4,(1+s+t+s*t)/4,(1-s+t-s*t)/4];
   deriv = @(s,t)[(-1+t)/4,( 1-t)/4,( 1+t)/4,(-1-t)/4 ;
                  (-1+s)/4,(-1-s)/4,( 1+s)/4,( 1-s)/4 ];

    pospg = [ -0.577350269189626E+00 , 0.577350269189626E+00 ];
    pespg = [  1.0E+00 , 1.0E+00];

    strsg = [];
    extrap = [];
    order = [ 1 , 4 ; 2 , 3 ]; % Align the g-pts. with the element corners

    for i=1 : 2
       for j=1 : 2
          lcder = deriv(pospg(i),pospg(j)) ;    % FF Local derivatives
          xjacm = lcder*nodes ;                 % Jacobian matrix
          ctder = xjacm\lcder ;                 % FF Cartesian derivates

          bmat = [];
          for inode = 1 : 4
            bmat = [ bmat , [ctder(1,inode),            0 ;
                                        0 ,ctder(2,inode);
                             ctder(2,inode),ctder(1,inode) ] ] ;
          end

          strsg(:,order(i,j)) = (dmat*bmat*displ) ;

          a = 1/pospg(i);
          b = 1/pospg(j);
```

```
          extrap(order(i,j),:) = fform(a,b) ;
        end
     end

     se = transpose(extrap*transpose(strsg));

%  Plane Stress
   if (pstrs==1)
        S = se ;
%  Plane Strain
    else
        S = [se(1,:) ; se(2,:) ; -poiss*(se(1,:)+se(2,:)) ; se(3,:)];
    end
```

I. Función ErrEst.m

```
function [E,U] = ErrEst (dmat,thick,pstrs,Strnod,Struns)

%% ErrEst Estima el error y calcula la energía de deformación en cada
%         elemento por separado.
%
%  Parámetros:
%
%    Input, dmat: matriz constitutiva
%           thick: espesor
%           pstrs: indicador para Deformación Plana
%          Strnod: tensiones en los nodos
%          Struns: tensiones sin alisar en los nodos
%
%    Output, E el vector del error estimado en los elementos
%            U el vector de la energía de deformación de los elemento
%
   global coordinates;
   global elements;

   nelem  = size(elements,1);        % Número de elementos
   nnode  = size(elements,2);        % Número de nodos por elemento

   invdmat = inv(dmat);              % Inversa de la matriz constitutiva
   E = [];
   U = [];

   if pstrs == 0     % Deformación Plana
       SN = [Strnod(:,1) Strnod(:,2) Strnod(:,4)];
       ElemStrCte = [Struns(:,1) Struns(:,2) Struns(:,4)];
   else              % Tensión Plana
       SN = Strnod;
       ElemStrCte = Struns;
   end

   % Elementos triangulares
   if nnode == 3
       for ielem = 1 : nelem
           lnods = elements(ielem,:);
           nodes(1:nnode,:) = coordinates(lnods(1:nnode),:);
           % Cálculo del volumen
           b(1) = nodes(2,2) - nodes(3,2);
           b(2) = nodes(3,2) - nodes(1,2);
           c(1) = nodes(3,1) - nodes(2,1);
```

```
        c(2) = nodes(1,1) - nodes(3,1);
        vol = (abs(b(1)*c(2) - b(2)*c(1)))/2*thick;
        % Estimación del error y energía de deformación
        ElemStrSmo = (SN(lnods(1),:)+SN(lnods(2),:)+SN(lnods(3),:))/3;
        ElemStrDif = ElemStrSmo - ElemStrCte(ielem,:);
        E=[E;sqrt(ElemStrDif*invdmat*transpose(ElemStrDif)*vol)];
        U=[U;sqrt(ElemStrSmo*invdmat*transpose(ElemStrSmo)*vol)];
    end

% Elementos cuadriláteros
else
% Definiciones previas
fform = @(s,t)[(1-s-t+s*t)/4,(1+s-t-s*t)/4,(1+s+t+s*t)/4,(1-s+t-s*t)/4];
deriv = @(s,t)[(-1+t)/4,( 1-t)/4,( 1+t)/4,(-1-t)/4 ;
               (-1+s)/4,(-1-s)/4,( 1+s)/4,( 1-s)/4 ];
pospg = [ -0.577350269189626E+00 , 0.577350269189626E+00 ];
pespg = [  1.0E+00 , 1.0E+00];
extrap = [];              % Matriz de extrapolación
order = [ 1 , 4 ; 2 , 3 ];% Alineación de los pts. de gauss con los nodos
for i = 1 : 2
    for j = 1 : 2
        a = 1/pospg(i);
        b = 1/pospg(j);
        extrap(order(i,j),:) = fform(a,b);
    end
end
invextrap = inv(extrap);  % Inversa de la matriz de extrapolación
% Fin de las definiciones previas
    for ielem = 1 : nelem
        lnods = elements(ielem,:);
        nodes(1:nnode,:) = coordinates(lnods(1:nnode),:);
        NodeStrSmo = [];
        NodeStrDif = [];
        % Cálculo de los volúmenes asignados a cada punto de gauss
        dvol = [];
        for i = 1 : 2
            for j = 1 : 2
                lcder = deriv(pospg(i),pospg(j)); % Derivadas FF locales
                xjacm = lcder*nodes ;             % Matriz Jacobiana
                dvol = [dvol;det(xjacm)*pespg(i)*pespg(j)*thick];
            end
        end
        % Tensiones en los nodos
        for i = 1 : 4
            NodeStrSmo(i,:) = SN(lnods(i),:);
            NodeStrDif(i,:) = NodeStrSmo(i,:) - ...
                              ElemStrCte((4*(ielem-1)+i),:);
        end
        % Tensiones en los puntos de gauss
        GaussStrSmo = invextrap*NodeStrSmo;
        GaussStrDif = invextrap*NodeStrDif;
        % Estimación del error y energía de deformación
        auxee = [];
        auxeu = [];
        for i = 1 : 4
            auxee(i) = GaussStrDif(i,:)*invdmat*...
                       transpose(GaussStrDif(i,:))*dvol(i);
            auxeu(i) = GaussStrSmo(i,:)*invdmat*...
                       transpose(GaussStrSmo(i,:))*dvol(i);
        end
        E = [E;sqrt(sum(auxee))];
        U = [U;sqrt(sum(auxeu))];
    end
end
```

J. Función Refinement.m

```
function [ERP,NES] = Refinement (cmo,EError,GlobalError,maxerror,...
                                 GlobalU,EVol,ElemDim)

%% Refinement Determina los parámetros de refinamiento de cada elemento y
%             define los nuevos tamaños de los elementos.
%
%  Parámetros:
%
%    Input, cmo: criterio de malla óptima elegido
%                1. Equi-distribución del error global
%                2. Equi-distribución del error específico
%         EError: vector del error estimado en cada elemento
%   GlobalError: error global estimado
%      maxerror: error máximo permitido
%       GlobalU: energía de deformación global
%          EVol: vector de volúmenes de elemento
%       ElemDim: vector de tamaños de elemento
%
%    Output, ERP el vector de parámetros de refinamiento de los elementos
%            NES el vector de nuevos tamaños de elemento
%
  global elements;

  nelem  = size(elements,1);           % Número de elementos
  nnode  = size(elements,2);           % Número de nodos por elemento

  ERP = [];
  NES = [];
  m = 1;                % Grado del polinomio de las funciones de forma
  d = 2;                % Dimensiones del problema
  q = (2*m+d)/2;        % Tasa de convergencia del parámetro de
                        % refinamiento local con EDEGlobal
  TotalVolume = sum(EVol);
  for i = 1 : nelem
      if cmo == 1       % Equidistribucion del error global
          ERP = [ERP;(EError(i)*sqrt(nelem)/GlobalError)^(1/q)*...
                            (GlobalError/(maxerror*GlobalU))^(1/m)];
      elseif cmo == 2   % Equi-distribucion del error especifico
          ERP = [ERP;((EError(i)/(maxerror*GlobalU))*...
                             sqrt(TotalVolume/EVol(i)))^(1/m)];
      end
      NES = [NES;ElemDim(i)/ERP(i)];
  end
```

K. Función ToGiD.m

```
function ToGiD (file_name,u,reaction,Strnod,ElemRef,NewElemSize)

%% ToGiD Writes the postprocess files and the new mesh file.
%
%  Parameters:
%
%    Input,  u           : Nodal displacements
%            reaction    : Reactions on fixed nodes
%            Strnod      : Nodal Stresses
%            ElemRef     : Element Refinement Parameter
%            NewElemSize : New Element Sizes
%
%    Output, none
%
  global coordinates;
  global elements;

  nelem  = size(elements,1);           % Number of elements
  nnode  = size(elements,2);           % Number of nodes per element
  npnod  = size(coordinates,1);        % Number of nodes

  if (nnode == 3)
    eletyp = 'Triangle';
    eletypbgm = 'Triangle';
  else
    eletyp = 'Quadrilateral';
    eletypbgm = 'Tetrahedra';
  end

  msh_file = strcat(file_name,'.flavia.msh');
  res_file = strcat(file_name,'.flavia.res');
  bgm_file = strcat(file_name,'.bgm');

% Mesh File
  fid = fopen(msh_file,'w');
  fprintf(fid,'### \n');
  fprintf(fid,'# MAT_FEM  V.1.0 \n');
  fprintf(fid,'# \n');
  fprintf(fid,['MESH dimension %3.0f   Elemtype %s   Nnode %2.0f \n
\n'],2,eletyp,nnode);
  fprintf(fid,['coordinates \n']);
  for i = 1 : npnod
    fprintf(fid,['%6.0f %12.5d %12.5d \n'],i,coordinates(i,:));
  end
  fprintf(fid,['end coordinates \n \n']);
  fprintf(fid,['elements \n']);
  if (nnode == 3)
    for i = 1 : nelem
      fprintf(fid,['%6.0f %6.0f %6.0f %6.0f   1 \n'],i,elements(i,:));
    end
  else
    for i = 1 : nelem
      fprintf(fid,['%6.0f %6.0f %6.0f %6.0f %6.0f  1 \n'],i,elements(i,:));
    end
  end
  fprintf(fid,['end elements \n \n']);

  status = fclose(fid);

% Results File
  fid = fopen(res_file,'w');
  fprintf(fid,'Gid Post Results File 1.0 \n');
  fprintf(fid,'### \n');
  fprintf(fid,'# MAT_FEM  V.1.0 \n');
```

```
  % Displacements
  fprintf(fid,'# \n');
  fprintf(fid,['Result "Displacements" "Load Analysis"  1  Vector OnNodes
\n']);
  fprintf(fid,['ComponentNames "X-Displ", "Y-Displ", "Z-Displ" \n']);
  fprintf(fid,['Values \n']);
  for i = 1 : npnod
    fprintf(fid,['%6.0i %13.5d %13.5d \n'],i,full(u(i*2-1)),full(u(i*2)));
  end
  fprintf(fid,['End Values \n']);
  % Reactions
  fprintf(fid,'# \n');
  fprintf(fid,['Result "Reaction" "Load Analysis"  1  Vector OnNodes \n']);
  fprintf(fid,['ComponentNames "Rx", "Ry", "Rz" \n']);
  fprintf(fid,['Values \n']);
  for i = 1 : npnod
    fprintf(fid,['%6.0f %12.5d %12.5d \n'],i,full(reaction(i*2-
1)),full(reaction(i*2)));
  end
  fprintf(fid,['End Values \n']);
  % Stresses
  fprintf(fid,'# \n');
  fprintf(fid,['Result "Stresses" "Load Analysis"  1  Matrix OnNodes \n']);
  fprintf(fid,['ComponentNames "Sx", "Sy", "Sz", "Sxy", "Syz", "Sxz" \n']);
  fprintf(fid,['Values \n']);

  if size(Strnod,2) == 3
    for i = 1 : npnod
      fprintf(fid,['%6.0f %12.5d %12.5d  0.0 %12.5d  0.0  0.0
\n'],i,Strnod(i,:));
    end
  else
    for i = 1 : npnod
      fprintf(fid,['%6.0f %12.5d %12.5d %12.5d %12.5d  0.0  0.0
\n'],i,Strnod(i,:));
    end
  end
  fprintf(fid,['End Values \n']);
  %%% Definición de los Puntos de Gauss
  fprintf(fid,'GaussPoints "Elements" Elemtype %s \n',eletyp);
  fprintf(fid,'Number of Gauss Points: 1 \n');
  fprintf(fid,'Natural Coordinates: Internal \n');
  fprintf(fid,'End GaussPoints \n');
  %%%
  %%% Parámetros de Refinamiento de los Elementos
  fprintf(fid,'# \n');
  fprintf(fid,['Result "Refinement" "Load Analysis"  1  Scalar OnGaussPoints
Elements \n']);
  fprintf(fid,['ComponentNames "Refinement_Parameter" \n']);
  fprintf(fid,['Values \n']);
  for i = 1 : nelem
    fprintf(fid,['%6.0f %12.5d \n'],i,ElemRef(i));
  end
  fprintf(fid,['End Values \n']);
  %%%
  status = fclose(fid);

  %%% Creación de la background mesh con los nuevos tamaños de elemento
  fid = fopen(bgm_file,'w');
  fprintf(fid,'BackgroundMesh V 1.0\n');
  fprintf(fid,'MESH dimension %0.0f ElemType %s Nnode
%0.0f\n',2,eletypbgm,nnode);
  fprintf(fid,'Coordinates\n');
  for i = 1 : npnod
      fprintf(fid,['%6.0f %12.5d %12.5d \n'],i,coordinates(i,:));
  end
  fprintf(fid,'end coordinates\n\n');
  fprintf(fid,'Elements\n');
```

```
  if (nnode == 3)
      for i = 1 : nelem
          fprintf(fid,['%6.0f %6.0f %6.0f %6.0f\n'],i,elements(i,:));
      end
  else
      for i = 1 : nelem
          fprintf(fid,['%6.0f %6.0f %6.0f %6.0f %6.0f  1
\n'],i,elements(i,:));
      end
  end
  fprintf(fid,'end elements\n\n');
  fprintf(fid,'DesiredSize Elements\n');
  for i = 1 : nelem
      fprintf(fid,'%0.0f %0.5f\n',i,NewElemSize(i,1));
  end
  fprintf(fid,'End DesiredSize\n');
  status = fclose(fid);
  %%%
```